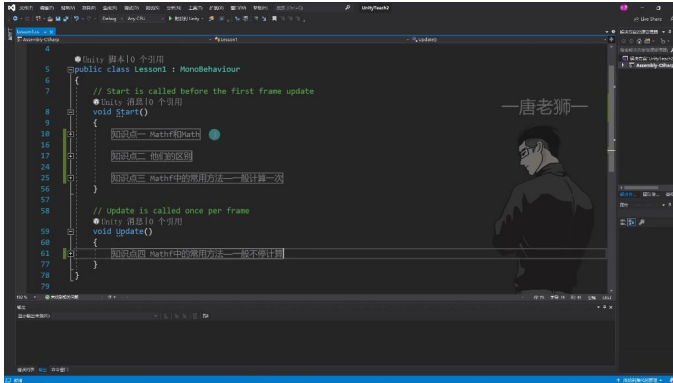


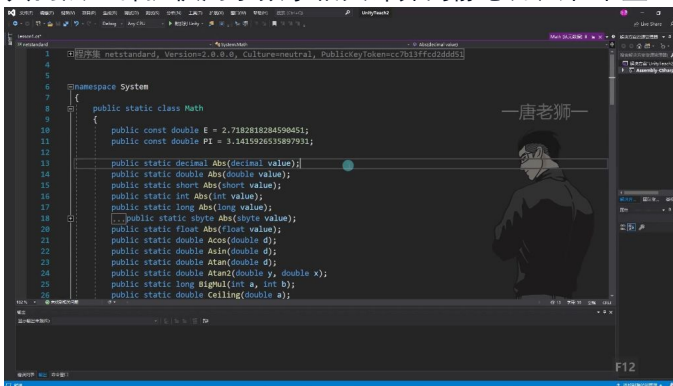
以下为AI生成的图文笔记的内容

一、数学计算公共类Mathf 00:04

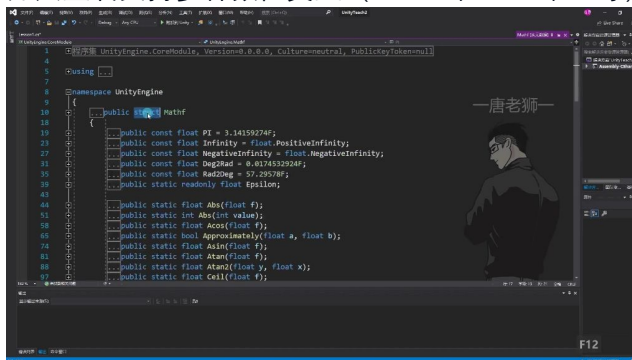
1. 知识点一 Mathf和Math 01:10



-
- **Math工具类**：C#中封装好的用于数学计算的工具类，位于System命名空间中
- **Mathf结构体**：Unity中封装好的用于数学计算的工具结构体，位于UnityEngine命名空间中
- **共同点**：都提供用于数学相关计算的静态方法和常量

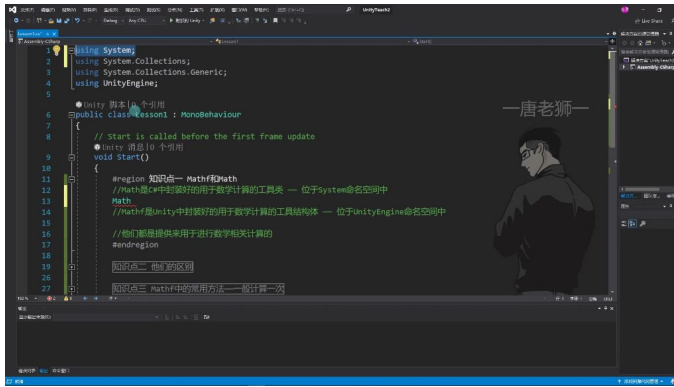


-
- **Math类特性**：
 - 包含常用数学常量如 $E = 2.7182818284590451$ 和 $PI = 3.1415926535897931$
 - 提供绝对值计算、三角函数、对数运算等静态方法
 - 方法重载支持多种数值类型（decimal, double, int等）



-
- **Mathf特性**：
 - 包含Unity专用的数学常量如 $PI = 3.14159274F$
 - 提供角度弧度转换常量 $Deg2Rad = 0.0174532924F$ 和 $Rad2Deg = 57.29578F$
 - 包含游戏开发专用方法如Lerp插值、平滑阻尼等

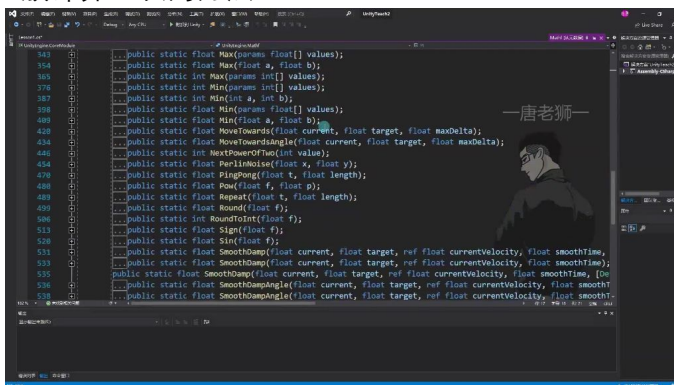
2. 知识点二 他们的区别



-
- 来源区别：
 - Math属于C#基础类库，位于System命名空间
 - Mathf是Unity引擎特有，位于UnityEngine命名空间
- 类型区别：
 - Math是静态类（static class）
 - Mathf是结构体（struct）
- 功能区别：
 - Math提供基础数学运算功能
 - Mathf额外包含游戏开发专用方法

3. 知识点三 Mathf中的常用方法

1) 一般计算一次的方法



-
- 基本运算：
 - Abs(): 计算绝对值
 - Ceil()/Floor(): 向上/向下取整
 - Round(): 四舍五入
- 三角函数：
 - Sin()/Cos(): 正弦/余弦计算
 - Atan2(): 计算反正切值
- 特殊计算：
 - Clamp(): 数值范围限制
 - Lerp(): 线性插值计算

2) 一般不停计算的方法



The image shows a screenshot of the Unity development environment. The main window displays a C# script named 'Test' with the following code and comments:

```
using UnityEngine;

// 1. Mathf 是 Unity 的一个引用
void Start()
{
    // 知识点 1 Math和Mathf
    // Mathf是Unity中封装好用于数学计算的工具类 —— 位于System命名空间中

    // Mathf是Unity中封装好的用于数学计算的的工具结构体 —— 位于UnityEngine命名空间中

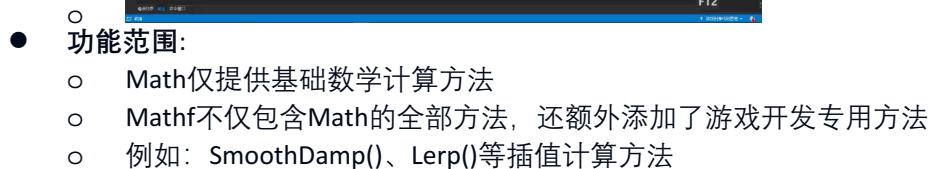
    // 他们都被提供用于进行数学相关计算的
}

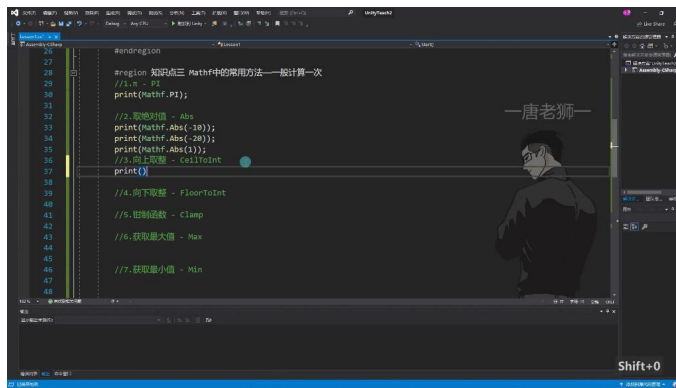
// 2. Vector3 是 Unity 的一个引用
void Update()
{
    // 知识点二 他们的区别
    // Mathf 和 Mathf 中的调用方式几乎一样
    // Vector3 是 Unity 中的工具类，主要提供了一些数学相关计算方法
    // Mathf 是 Unity 中封装好的，不包含在 Mathf 中的方法，还提供了一些适用于游戏开发的方法
    // 所以我在进行 Unity 游戏开发时
    // 用到 Mathf 中的方法用于数学计算即可
}

// 知识点三 Mathf 中的调用方式 —— 做计算一次
// Update is called once per frame
```

On the right side of the script editor, there is a cartoon character of a man with glasses and a beard, wearing a dark jacket, standing with his hands in his pockets. The character is labeled '唐老师' (Teacher Tang) in Chinese.

- Math是C#自带的数学工具类，位于System命名空间
- Mathf是Unity封装的数学工具结构体，位于UnityEngine命名空间

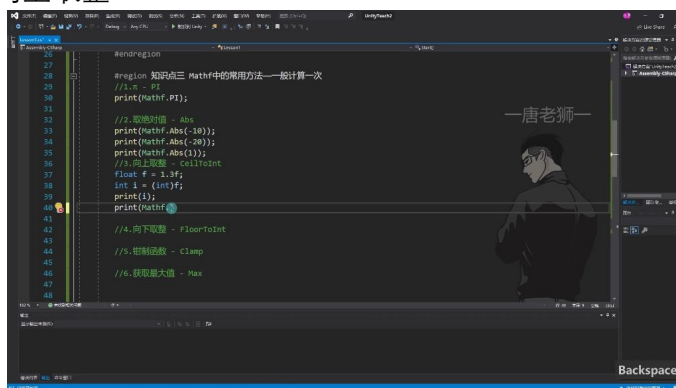




```
27 #endregion
28 #region 知识点三 Mathf中的常用方法——取绝对值
29 //1. 取绝对值 - Abs
30 print(Mathf.Abs(-10));
31 //2. 取绝对值 - Abs
32 print(Mathf.Abs(-10));
33 print(Mathf.Abs(-20));
34 print(Mathf.Abs(1));
35 //3. 向上取整 - CeilToInt
36 print(0);
37
38 //4. 向下取整 - FloorToInt
39
40 //5. 限制范围 - Clamp
41
42 //6. 获取最大值 - Max
43
44 //7. 获取最小值 - Min
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Shift+O

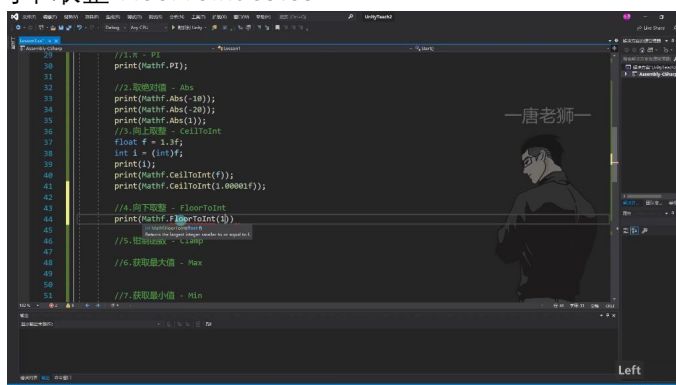
- - 功能: 返回数值的绝对值（正数形式）
 - 数学定义: 负数的绝对值是其对应的正数, 如 $|-10|=10$
 - 重载方法:
 - Mathf.Abs(float f)
 - Mathf.Abs(int value)
 - 示例:
- 3) 向上取整-CeilToInt 07:19



```
27 #endregion
28 #region 知识点三 Mathf中的常用方法——取绝对值
29 //1. 取绝对值 - Abs
30 print(Mathf.Abs(-10));
31 //2. 取绝对值 - Abs
32 print(Mathf.Abs(-10));
33 print(Mathf.Abs(-20));
34 print(Mathf.Abs(1));
35 //3. 向上取整 - CeilToInt
36 float f = 1.3f;
37 int i = (int)f;
38 print(i);
39 print(Mathf.CeilToInt(f));
40
41 //4. 向下取整 - FloorToInt
42
43 //5. 限制范围 - Clamp
44
45 //6. 获取最大值 - Max
46
47 //7. 获取最小值 - Min
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Backspace

- - 功能: 将浮点数向上取整到最近的整数
 - 与强制转换区别:
 - 强制转换(int)f会向下取整
 - CeilToInt始终向上取整
 - 特性: 只要小数部分>0就会进位
 - 示例:
- 4) 向下取整-FloorToInt 09:09



```
27 #endregion
28 #region 知识点三 Mathf中的常用方法——取绝对值
29 //1. 取绝对值 - Abs
30 print(Mathf.Abs(-10));
31 //2. 取绝对值 - Abs
32 print(Mathf.Abs(-10));
33 print(Mathf.Abs(-20));
34 print(Mathf.Abs(1));
35 //3. 向上取整 - CeilToInt
36 float f = 1.3f;
37 int i = (int)f;
38 print(i);
39 print(Mathf.CeilToInt(f));
40 print(Mathf.CeilToInt(1.00001f));
41
42 //4. 向下取整 - FloorToInt
43 print(Mathf.FloorToInt(0));
44
45 //5. 限制范围 - Clamp
46
47 //6. 获取最大值 - Max
48
49 //7. 获取最小值 - Min
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Left

-
- 功能: 将浮点数向下取整到最近的整数
- 与强制转换区别:
 - 结果与强制转换(int)f相同
 - 但语法更简洁直观
- 特性: 直接舍去小数部分

- 示例:

5) 钳制函数-Clamp

```

//region 知识点三 Mathf中的常用方法——一般计算一次
//1. 取 - PI
print(Mathf.PI);

//2. 取绝对值 - Abs
print(Mathf.Abs(-10));
print(Mathf.Abs(-20));
print(Mathf.Abs(1));

//3. 向上取整 - CeilToInt
print(Mathf.CeilToInt(1.3f));

//4. 向下取整 - FloorToInt
print(Mathf.FloorToInt(9.6f));

//5. 钳制函数 - Clamp
print(Mathf.Clamp(10, 11, 20));
print(Mathf.Clamp(21, 11, 20));
print(Mathf.Clamp(15, 11, 20));

//6. 获取最大值 - Max
print(Mathf.Max(10, 11, 20));

//7. 获取最小值 - Min
print(Mathf.Min(10, 11, 20));

//8. 一个数的n次幂 - Pow
print(Mathf.Pow(2, 10));

```

- 功能: 将数值限制在指定范围内
- 参数: Clamp(float value, float min, float max)
- 返回值规则:
 - 若value在[min,max]区间内, 返回value
 - 若value<min, 返回min
 - 若value>max, 返回max

6. 知识点四 10:08

1) Mathf中的常用方法 10:11

- 一般计算一次的方法

```

print(Mathf.Abs(1));
//3. 向上取整 - CeilToInt
float f = 1.3f;
int i = (int)f;
print(i);
print(Mathf.CeilToInt(f));
print(Mathf.CeilToInt(1.00001f));

//4. 向下取整 - FloorToInt
print(Mathf.FloorToInt(9.6f));

//5. 钳制函数 - Clamp
print(Mathf.Clamp(10, 11, 20));
print(Mathf.Clamp(21, 11, 20));
print(Mathf.Clamp(15, 11, 20));

//6. 获取最大值 - Max
print(Mathf.Max(10, 11, 20));

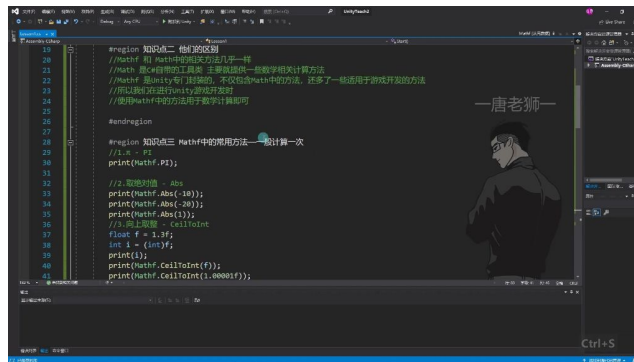
//7. 获取最小值 - Min
print(Mathf.Min(10, 11, 20));

//8. 一个数的n次幂 - Pow
print(Mathf.Pow(2, 10));

```

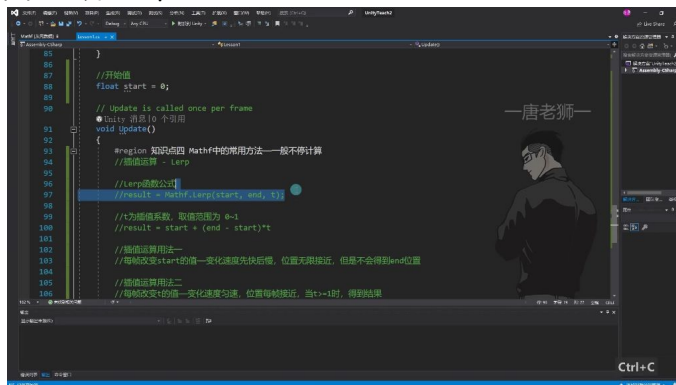
- 钳制函数(Clamp):
 - 别名"夹紧函数"或"前置函数", 将数值限制在最小值和最大值之间
 - 规则: 比最小值小取最小值, 比最大值大取最大值, 在区间内取原值
 - 示例: Mathf.Clamp(10,11,20)返回11, Mathf.Clamp(21,11,20)返回20, Mathf.Clamp(15,11,20)返回15
- 向上取整(CeilToInt):
 - 对浮点数向上取整为整数
 - 示例: Mathf.CeilToInt(1.3f)返回2, Mathf.CeilToInt(1.00001f)返回2
- 向下取整(FloorToInt):
 - 对浮点数向下取整为整数
 - 示例: Mathf.FloorToInt(9.6f)返回9
- 获取极值方法

- **判断2的n次方(IsPowerOfTwo):**
 - 判断一个数是否是2的幂次方
 - 原理: 基于二进制表示特性
 - 示例: `Mathf.IsPowerOfTwo(4)`返回true, `Mathf.IsPowerOfTwo(8)`返回true, `Mathf.IsPowerOfTwo(3)`返回false
- **判断正负数(Sign):**
 - 返回数值的符号
 - 规则: 正数返回1, 负数返回-1, 零返回0
 - 示例: `Mathf.Sign(10)`返回1, `Mathf.Sign(-10)`返回-1, `Mathf.Sign(0)`返回0
- **Mathf与Math的区别**



- **来源差异:**
 - Math是C#自带的数学工具类
 - Mathf是Unity专门封装的数学工具类
- **功能差异:**
 - Mathf包含Math中的方法
 - Mathf额外提供适用于游戏开发的方法
- **使用建议:**
 - Unity开发中优先使用Mathf

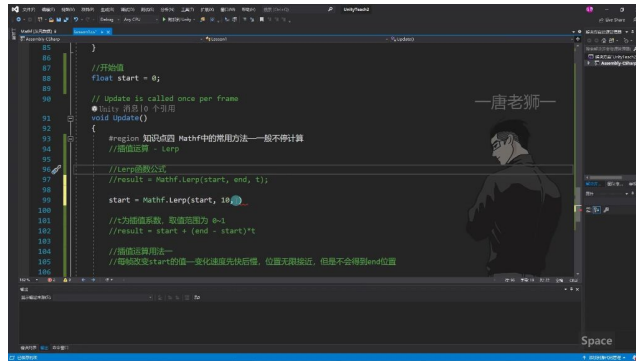
2) 插值运算-Lerp 20:41



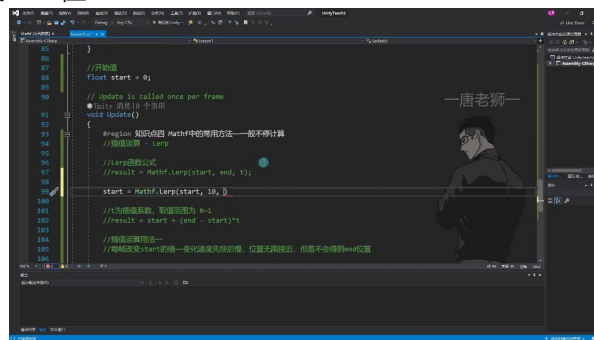
- **定义:** 差值运算是指通过Mathf.Lerp函数在两个值之间进行平滑过渡的计算方法
- **特点:** 这是Unity特有的数学运算方法, 传统数学课程中未曾涉及
- **应用场景:** 主要用于游戏开发中物体的平滑移动、颜色渐变等需要渐进变化的场景

3) Lerp函数公式 21:02

- **基本公式**

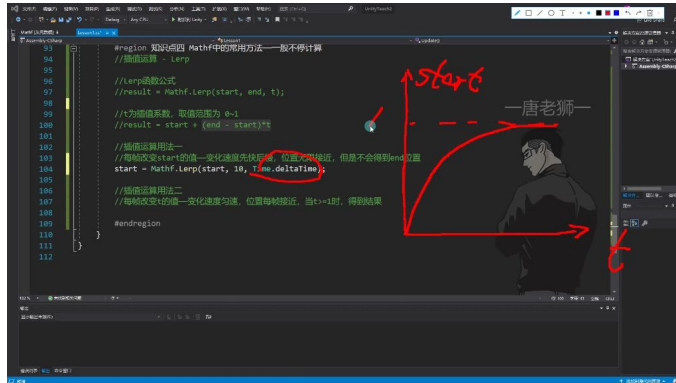


- 标准形式: $result = Mathf.Lerp(start, end, t)$
- 展开公式: $result = start + (end - start) * t$
- 参数说明:
 - start: 起始值 (如示例中的0)
 - end: 结束值 (如示例中的10)
 - t: 插值系数, 取值范围为0~1
- 参数特性
 - t值范围: 必须介于0到1之间, 常用Time.deltaTime这样的小数值作为增量
 - 计算原理: 结果值是基于起始值与目标值之间的比例位置
 - 数学意义: 当t=0时结果为start, t=1时结果为end, 中间值按比例线性插值
- 两种用法
 - 改变start值



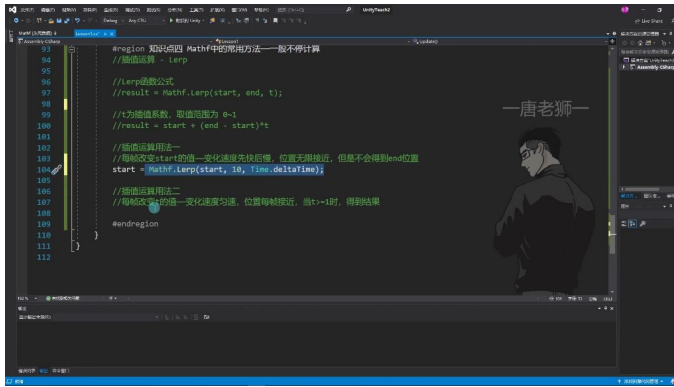
- 实现方式: 每帧更新start值, 如 $start = Mathf.Lerp(start, end, t)$
- 运动特点:
 - 速度变化: 先快后慢
 - 最终状态: 无限接近end但理论上永远不会达到
 - 适用场景: 需要渐进减速的效果
- 改变t值
 - 实现方式: 保持start不变, 逐步增加t值
 - 运动特点:
 - 速度变化: 保持匀速
 - 最终状态: 当 $t \geq 1$ 时可精确到达end位置
 - 适用场景: 需要线性变化的效果
- 计算示例
 - 示例参数:
 - start = 0
 - end = 10
 - t = 0.1
 - 计算结果: $0 + (10 - 0) * 0.1 = 1$
 - 实际应用: 在Update()中每帧执行, 可实现平滑过渡效果

4) 插值运算用法一 22:20

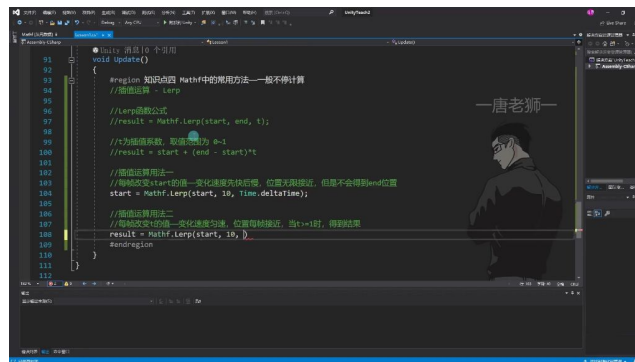


- 基本公式
 - 函数原型: `result = Mathf.Lerp(start, end, t);`
 - 数学等价: $result = start + (end - start) \times t$
 - 参数说明:
 - `t`为插值系数，取值范围[0,1]
 - `start`为起始值，`end`为目标值
- 动态特性
 - 变化规律:
 - 当每帧更新`start`值时（示例代码: `start = Mathf.Lerp(start, 10, Time.deltaTime);`）
 - 变化速度呈现先快后慢的特征
 - 结果值会无限趋近于目标值但永不相等
 - 数学原理:
 - 首次计算: $0 + (10 - 0) \times t = 10t$
 - 后续计算: 因`start`值持续更新， $(end - start)$ 差值逐渐减小
 - 导致增量部分 $(end - start) \times t$ 逐帧递减
- 可视化解理解
 - 坐标系演示:
 - 横轴表示时间变化
 - 纵轴表示`start`值变化
 - 曲线特征: 初始斜率大（变化快），后期渐缓（变化慢）
 - 极限特性:
 - 当 $t \rightarrow \infty$ 时， $start \rightarrow end$ （数学极限）
 - 实际编程中由于浮点精度限制，最终会稳定在极接近`end`的值
- 典型应用场景
 - 适用情况:
 - 需要平滑过渡的效果（如摄像机跟随）
 - 对最终精度要求不严格的渐进变化
 - 注意事项:
 - 不能用于必须精确到达目标值的场景
 - 时间参数`t`建议使用`Time.deltaTime`保证帧率无关

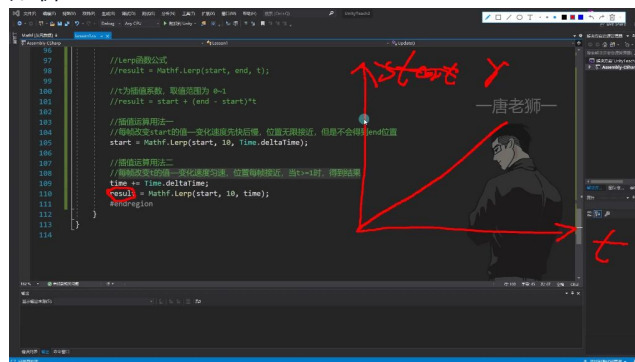
5) 插值运算用法二 25:24



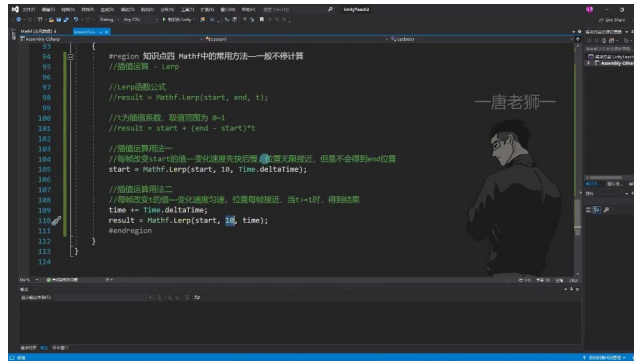
- 基本公式: $result = Mathf.Lerp(start, end, t)$ 或 $result = start + (end - start) * t$
- 参数说明:
 - start: 起始值
 - end: 目标值
 - t: 插值系数，取值范围为 0~1
- 实现方式



- 变量声明:
- 更新逻辑:
- 特性分析



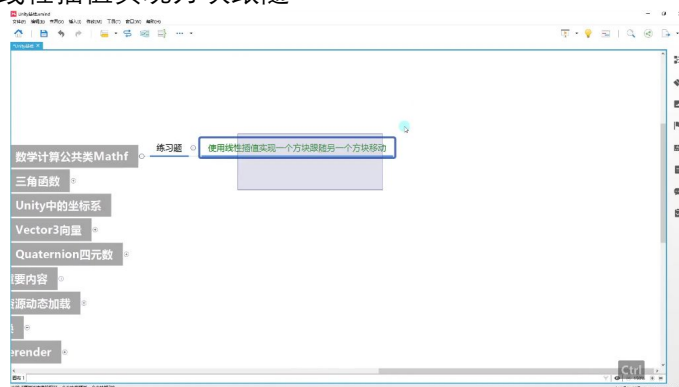
- 运动特性:
 - 匀速变化: 随时间均匀接近目标值
 - 精确到达: 当 $t \geq 1$ 时，结果会精确等于 end 值
- 数学原理:
 - 由于 start 保持不变，公式简化为 $0 + (10 - 0) * t$
 - 每帧增加相同的 $Time.deltaTime$ ，导致变化量均匀
- 与用法一的区别



-
- **变化速度:**
 - 用法一: 先快后慢
 - 用法二: 匀速变化
- **最终结果:**
 - 用法一: 无限接近但永不等于end
 - 用法二: 最终会精确等于end
- **参数变化:**
 - 用法一: 改变start值
 - 用法二: 改变t值
- **实际应用**
 - **跟随移动:** 实现对象A平滑跟随对象B的效果
 - 将对象B的坐标作为end值
 - 每帧计算对象A的新位置
 - **值趋近:** 任何需要值平滑过渡的场景
 - 如UI透明度变化
 - 摄像机平滑移动
- **记忆要点**
 - **核心区别:**
 - 变start→先快后慢不达终值
 - 变t→匀速变化可达终值
 - **应用场景选择:**
 - 需要精确到达时用用法二
 - 需要阻尼效果时用用法一

7. 应用案例 30:11

1) 线性插值实现方块跟随



-
- **实现方法:** 使用Unity中的线性插值函数实现一个方块跟随另一个方块移动的效果
- **数学基础:** 需要掌握Mathf类中的数学计算方法
- **坐标系转换:** 理解Unity中的Vector3向量和Quaternion四元数坐标系
- **动态加载:** 通过代码动态控制游戏对象的渲染和移动

2) 练习题：线性插值应用



- 练习要求：
 - 使用线性插值函数实现方块跟随效果
 - 可截图保存制作过程
 - 将在下个视频中讲解实现方法
- 技术要点：
 - 掌握Mathf.Lerp函数的参数设置
 - 理解插值系数对移动效果的影响
 - 注意坐标系转换的准确性

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
Mathf与Math的区别	Math是C#自带的数学计算工具类（System命名空间），Mathf是Unity封装的数学计算结构体（UnityEngine命名空间）， Mathf包含更多游戏开发专用方法。	Math是类，Mathf是结构体；Mathf方法更全面，优先使用。	★★
常用静态变量	Mathf.PI（圆周率常量）	直接调用，无需手动定义。	★
绝对值计算	Mathf.Abs(-10) → 10	负数的绝对值是其正数形式。	★
取整方法	- Mathf.CeilToInt(1.3f) → 向上取整为2 - Mathf.FloorToInt(1.6f) → 向下取整为1	非四舍五入，与强制类型转换的区别。	★★
钳制函数 (Clamp)	Mathf.Clamp(10, 11, 20) → 11（超出范围取边界值）	比最小值小取最小，比最大值大取最大，否则取原值。	★★
极值计算	- Mathf.Max(1,2,8) → 8 - Mathf.Min(0.5f, 0.4f) → 0.4	支持多参数输入。	★
幂与平方根	- Mathf.Pow(4, 2) → 16（4的2次方）	幂运算需区分底数与指数。	★★

	- Mathf.Sqrt(16) → 4		
二的幂判断	Mathf.IsPowerOfTwo(4) → true	用于二进制位运算场景。	★★
符号判断	Mathf.Sign(-10) → -1 (负数返回-1, 正数返回1)	零被视为正数。	★
线性插值 (Lerp)	- 用法1: start = Mathf.Lerp(start, end, t) → 先快后慢, 无限接近但不等于end - 用法2: result = Mathf.Lerp(0, 10, time) → 匀速变化, time ≥ 1时等于end	公式: start + (end - start) * t; 游戏对象缓动跟随的常用实现方式。	★★★